

Practical: 4

Implementation and Time analysis of factorial program using iterative and recursive method

Aim: To Implement and Time analysis of factorial program using iterative and recursive method

Algorithm:

Input :- n (some positive integer value)
output :- $n!$ ($n \times (n-1) \times (n-2) \dots \times 1$)

steps

step ① :- set fact = 1 and n = some value

step ② :- for $i = 1$ till $i \leq n$

Step ③ :- fact = fact * i

Step ④ :- end for

step ⑤ :- return fact

Program:

Iterative Code

```
#include<iostream>
using namespace std;

int factorial(int n){
    int fact = 1;

    for(int i = 1; i <= n; i++){
        fact = fact * i;
    }

    return fact;
}
```

```

    }

    int main(){
        int n;

        cout<<"enter the number ";
        cin>>n;

        cout<<"factorial of the number is: "<<factorial(n);

        return 0;
    }
    
```

Time Complexity:

for(int i=1; i<=n; i++) $\rightarrow n$ times
 fact=fact*i; $\rightarrow 1$ time
 $T(n) = n + 1 + 1$
 $T(n) = O(n)$

Analysis

Best case	Average	worst
$T(n) = 1 + (n+1) + n$ $= 2n + 2$ $T(n) = O(n)$ for $n=0$ $O(1)$	$T(n) = 2n + 2$ $T(n) = O(n)$	$T(n) = 2n + 2$ $T(n) = O(n)$

Output:

Iterative Method

```
Enter the number 5
Factorial of the number is: 120
-----
Process exited after 3.639 seconds with return value 0
Press any key to continue . . . |
```

Algorithm:

• Input: Element
output: Factorial of element

step ① :- start
step ② :- Given input n then do
step ③ :- factorial (n)
step ④ :- If $n == 0$ then action 1
step ⑤ :- Else action $n \times \text{factorial}(n-1)$
step ⑥ :- Display factorial
step ⑦ :- stop

Recursive Code

```
#include<iostream>
using namespace std;
```

```
int factorial(int n){
    if(n==0){
        return 1;    }
    else{
        return n*factorial(n-1);
    }
}
```

```
int main(){
```

```
int n;  
cout<<"enter the number ";  
cin>>n;  cout<<"factorial of the number is:  
"<<factorial(n);  
return 0;  
}
```

Recursive Method

```
Enter the number 6  
Recursive Method Factorial of the number is: 720  
-----  
Process exited after 1.667 seconds with return value 0  
Press any key to continue . . . |
```

Time Complexity:

```
if (n==0)  
    return 1;  
return n*factorial(n-1)  
for one recursive call  
T(n) = 1 + T(n-1)  
T(n) = T(n-1) + 1  
T(n) = T(n-1) + 1  
      = T(n-2) + 1 + 1  
      = T(0) + 2n  
since T(0) = 1  
T(n) = 2n + 1  
T(n) = O(n)
```

<u>Best case</u>	<u>Average</u>	<u>worst</u>
$T(n) = 1$	no. of calls = n	$T(n) = 2n + 2$
$T(n) = 2$	$T(n) = 2n + 2$	$T(n) = O(n)$
$T(n) = O(1)$	$T(n) = O(n)$	

Conclusion:

Hence, the Max-Heap Sort algorithm was successfully implemented and executed. It was observed that Heap Sort gives a time complexity of $O(n \log n)$ in the best, average, and worst cases. It is an efficient sorting technique and works well for sorting large sets of data.